<u>Aufgabe 1: UML Klassendiagramm</u> (15 Punkte, 15 min.)

Ein Carsharing-Anbieter stellt an mehreren Standorten Autos für seine Kunden zur Verfügung. Die Kunden können per Internet einen PKW für einen bestimmten Zeitraum buchen, und nach erfolgreicher Buchung mit ihrer elektronischen Kundenkarte das Auto aufschließen und benutzen.

Verwenden Sie folgenden Text zur Modellierung eines UML-Klassendiagramms, das die Verhältnisse des Carsharing-Anbieters darstellt:

> *Ein Carsharing-Anbieter besitzt mindestens einen Autostandort, wobei jeder Autostandort mindestens einen PKW besitzt. Ein Standort ist charakterisiert durch seine Adresse (Straße, PLZ, Stadt). PKWs können Kleinwagen, Mittelklassewagen oder Luxuslimousinen sein. Jeder PKW hat ein Kennzeichen, sowie einen Hersteller- und Typnamen. Außerdem besitzt jeder PKW eine eindeutige Fahrgestellnummer.*
> *Der Carsharing-Anbieter bietet seinen Kunden drei Verschiedene Vertragsmodelle an: "Basis", "Standard" und "Vielfahrer" sind Vertragsmodelle. Diese drei Vertragsmodelle haben alle unterschiedliche Werte für die km-Pauschale (€ pro Kilometer) und die Zeitpauschale (€ pro h). In Zukunft sollen noch weitere Vertragsmodelle leicht hinzuzufügen sein.*
> *Der Carsharing-Anbieter speichert für jeden seiner Kunden den Namen, das Geburtsdatum, die Führerscheinnummer und das Vertragsmodell.*
> *Eine Buchung hält fest, welcher Kunde in welchem Zeitraum (Starttermin, Endtermin) welchen konkreten PKW gebucht hat. Der Carsharing-Anbieter verwaltet eine beliebig große Anzahl von Buchungen, in schlechten Zeiten allerdings auch gar keine.*

Entwickeln Sie anhand des obigen Textes ein vollständiges UML-Klassendiagramm. Verwenden Sie sinnvolle Variablentypen für eventuelle Attribute. Verwenden Sie weiterhin geeignete Formen von Assoziationen zwischen den Klassen, und vergessen Sie nicht die Multiplizitäten.

Aufgabe 2: UML Sequenzdiagramm (15 Punkte, 15min.)

Gegeben ist der folgende Quelltext, der einen Auszug aus der Verwaltungs-Software des Carsharing-Anbieters darstellt:

```
import java.util.Date;
public class main {
        public static void main(String[] args) {
                Kunde kunde = new Kunde("Dicki Hoppenstedt", new Vertragsmodell_Basis());
                PKW   pkw   = new PKW  ("AC X 911");
                Date start = new Date();
                Date ende  = new Date();
                Verwaltung.Instance().reservieren(kunde, pkw, start, ende);
        }
}

public class Kunde {
        private String name;
        private Vertragsmodell vertrag;
        public Kunde(String n, Vertragsmodell v) {
                name    = n;
                vertrag = v;
        }
}

public class Vertragsmodell {
        public double km_pauschale;
        public double  h_pauschale;
}

public class Vertragsmodell_Basis extends Vertragsmodell {
        Vertragsmodell_Basis() {
                km_pauschale =  0.30;
                 h_pauschale = 10.00;
        }
}

public class PKW {
        private String kennzeichen;
        public PKW(String k) {
                kennzeichen = k;
        }
        public String getKennzeichen() {
                return kennzeichen;
        }
}

public class Verwaltung {
        // Singleton-Pattern
        private static Verwaltung _instance = null;
        private Verwaltung() {}
        public synchronized static Verwaltung Instance() {
                if (_instance == null) {
                        _instance = new Verwaltung();
                }
                return _instance;
        }

        public Buchung reservieren(Kunde k, PKW a, Date start, Date ende) {
                Buchung buchung = new Buchung();
                buchung.setKunde(k);
                buchung.setAuto (a);
                buchung.setStartZeit(start);
                buchung.setEndZeit(ende);
                return buchung;
        }
}

public class Buchung {

        private Date startZeit;
        private Date endZeit;
        private Kunde kunde;
        private PKW auto;
        public void setStartZeit(Date d) { startZeit = d; }
        public void setEndZeit(Date d)   { endZeit = d;   }
        public void setKunde(Kunde k)    { kunde = k;     }
        public void setAuto(PKW a)       { auto = a;      }

}
```

Normalerweise erstellen Sie Sequenzdiagramme im Rahmen eines objektorientierten Designs, aus dem sich dann Rückschlüsse auf den Code ableiten. In diesem Fall möchten Sie allerdings etwas Re-Engineering betreiben, und bereits existierenden Code mit UML-Diagrammen dokumentieren.
Erstellen Sie daher auf Basis des angegebenen Codes ein vollständiges UML-Sequenzdigramm für die Methode `main()`!

Hinweis: Aufrufe von statischen Methoden können mit einem "pseudo"-Objekt mit dem Namen `<static>:`*`Klassenname`* dargestellt werden.

<u>Aufgabe 3: UML Zustandsdiagramm</u> (15 Punkte, 15 min.)

Ein Funkwecker soll durch drei Tasten gesteuert werden:

1. Einen Ein-/Ausschalter für die Weckfunktion,
2. Einen Schlummerschalter: Wird der Schalter während des Weckens gedrückt, wird der Weckvorgang für 5 Minuten unterbrochen,
3. Einen Weckzeitwähler: durch Drücken des Schalters wird die aktuelle Weckzeit um 1 Minute vorgestellt. Eine Änderung der Weckzeit ist nur möglich, wenn der Wecker sich nicht im Weckvorgang befindet.

Beschreiben Sie mit einem Zustandsdiagramm die möglichen Zustände des Weckers und die jeweiligen Übergänge sowie die auslösenden Aktionen. Dabei wird der Wecker durch das Einsetzen der Batterie gestartet, beendet wird das Laufen des Weckers durch eine leere Batterie. Der Weckvorgang selber dauert maximal eine halbe Stunde, falls die Weckfunktion nicht vorher ausgeschaltet wurde. Danach wechselt der Wecker in seinen vorherigen Zustand.

<u>Aufgabe 4: Entwurfsmuster</u> (15 Punkte, 15 min.)

Ein Autowaschanlage bietet verschiedene Programme an:

a) Einfache Wäsche der Karosserie (5€)
b) Einfache Wäsche der Karosserie, Säuberung des Unterbodens (8€)
c) Einfache Wäsche der Karosserie, Säuberung des Unterbodens und Auftragen eines Schutzwachses (10,50€)
d) Einfache Wäsche der Karosserie, Säuberung des Unterbodens, Auftragen eines Schutzwachses und Felgenreinigung (12€)

Um das Angebot besser den einzelnen Kundenwünschen anpassen zu können, werden die einzelnen Elemente eines Waschvorgangs nun wahlfrei angeboten, so dass man sich einerseits eine ”punktgenaue“ Wäsche zusammenstellen kann, andererseits das Angebot flexibel erweitert werden kann, also neben der einfachen Wäsche der Karosserie auch eine Luxus-Wäsche der Karosserie angeboten werden kann.

Die Waschanlage soll nun softwareseitig aktualisiert werden, wobei sich für die Implementierung das Dekorierer-Muster anbietet.

1. Geben Sie das Klassendiagramm zu der zu entwickelnden Software auf Basis der oben geschilderten Informationen an *(8 Pkte.)*.

2. Vervollständigen Sie die folgende Implementation *(7 Pkte.)*:

```
Interface WaschElement
{
        public double preis();
        public String leistung();
}

class EinfacheWäsche

}

abstract class Dekorierer

        protected WaschElement element = null;

        Dekorierer(Waschelement a) { element = a; }
}

class Unterboden extends Dekorierer {

        Unterboden (WaschElement a) { super(a); }

}

class Wachs extends Dekorierer {

        Wachs(WaschElement a) { super(a); }

}

class Felgen extends Dekorierer {

        Felgen(WaschElement a) { super(a);      }

}


public static void main(String[] args) {

        Waschelement a = new EinfacheWäsche();
        a = new Unterboden( a );
        a = new Felgen( a );

        System.out.println(a.preis());
        System.out.println(a.leistung());
}
```

<u>Aufgabe 5: Testverfahren</u> (25 Punkte, 25min.)

Sie arbeiten als Qualitätsmanager bei einem Softwarehersteller. Ihre Aufgabe ist es, die nachfolgende Java-Funktion zu testen. Die Funktion sucht innerhalb des gegebenen Arrays **array** nach dem Wert **number**. Ist **number** im Array enthalten, gibt die Funktion **true** zurück, in allen anderen Fällen **false**.

Hinweise:

- **array** ist sortiert
- **array** enthält mindestens zwei Integer-Werte zwischen 1..99
- **number** ist vom Typ Integer

```
boolean search(int number, int[] array) {
   int i = 0;                                  /* K1 */
   int j = array.length;                       /* K2 */
   int mid = 0;                                /* K3 */
   boolean found = false;                      /* K4 */
   while ( (i < j) && !found ) {               /* K5 */
     mid = (i + j) / 2;                        /* K6 */
     found = true;                             /* K7 */
     if ( number < array[mid] ) {              /* K8 */
          j = mid - 1;                         /* K9 */
          found = false;                       /* K10 */
     }
     if ( number > array[mid] ) {              /* K11 */
          i = mid + 1;                         /* K12 */
          found = false;                       /* K13 */
     }
   }
   return found;                               /* K14 */
}
```

a) Zeichnen Sie den **Kontrollflußgraphen** der oben angegebenen Java-Funktion search(...). Fassen Sie dabei sequentiell aufeinanderfolgende Knoten so weit wie möglich zusammen, und beschriften Sie die Knoten und Kanten entsprechend! *(7 Punkte)*

b) Berechnen Sie die zyklomatische Komplexität CC dieser Funktion. *(3 Punkte)*

c) Betrachtet wird der Testfall `number =31, array={2,5,10,12,13,17,25,30,32}`
Berechnen Sie die C3-Überdeckung für diesen Fall mit Hilfe einer geeigneten Tabelle: *(10 Punkte)*

Testfall	`number=31, array={2,5,10,12,13,17,25,30,32}`
C3-Überdeckung:	

d) Begründen Sie, warum aus einer 100%-tigen C2-Überdeckung nicht automatisch auch eine Vollständige C1- und C0-Abdeckung resultiert.
(5 Punkte)

<u>Aufgabe 7: Verständnisfragen</u> (15 Punkte, 15 min.)

a) Warum ist die Entwicklung neuer Software schwieriger als anderer Produkte? Welche Eigenschaften von Software spielen hier eine Rolle? *(2 Punkte)*

b) Was ist der Unterschied zwischen konstruktiver und analytischer Qualitätssicherung? *(2 Punkte)*

c) Nennen Sie 4 Ziele für eine Software-Architektur. *(2 Punkte)*

d) Erklären Sie den Begriff der Kohäsion. *(2 Punkte)*

e) Wie funktioniert das Iterator-Pattern? *(2 Punkte)*

f) Was sind funktionale und nicht-funktionale Anforderungen und wie unterscheiden sie sich? Geben sie jeweils ein kurzes Beispiel an. *(2 Punkte)*

(bitte wenden)

g) Welche Gruppen von Design-Pattern gibt es? Beschreiben Sie diese kurz.
(3 Punkte)